

SigmaRL Traffic 观测空间组成说明

本文档专门说明当前项目中智能体观测空间的组成、维度、坐标系和归一化方式。

基于当前 `config.json` :

```
n_agents = 6
is_ego_view = true
is_partial_observation = true
n_nearing_agents_observed = 3
n_topology_nearing_agents_observed = 5
n_points_short_term = 3
is_observe_vertices = true
is_observe_distance_to_agents = true
is_observe_distance_to_boundaries = true
is_observe_distance_to_center_line = true
is_observe_ref_path_other_agents = false
is_using_opponent_modeling = true
is_add_noise = false
is_apply_mask = false
```

主要代码位置:

- `scenarios/road_traffic.py`
 - `observation()`
 - `_update_observation_and_normalize()`
 - `_observe_self()`
 - `_observe_other_agents()`
 - `info()`

1. 总体结论

当前每个 agent 的 actor 输入观测是:

49 维

张量形状:

```
("agents", "observation"): [B, 6, 49]
```

其中：

```
49 = 自车观测 10
    + 邻居观测 33
    + 对手建模动作占位 6
```

展开：

```
actor_obs = [
    self_observation,      # 10 维
    neighbors_observation, # 33 维
    opponent_action_tail,  # 6 维
]
```

2. 坐标系

当前：

```
is_ego_view = true
```

因此观测使用 ego 局部坐标系。

含义：

- 每个 agent 观察世界时，都以自己为坐标原点。
- x/y 方向相对于该 ego agent 的朝向。
- 邻居车辆的位置、顶点、速度、参考路径等都会被转换到 ego 坐标系。
- 自车全局位置和自车全局朝向不进入 actor 观测。

如果 `is_ego_view = false`，观测会使用全局坐标。

3. 自车观测

自车观测由 `_observe_self()` 生成。

当前自车观测维度：

D_self = 10

组成如下：

顺序	信息	维度	坐标系 / 说明
1	自车前向速度	1	ego 坐标系，只取前向速度分量
2	短期参考点 1	2	ego 坐标系， (x, y)
3	短期参考点 2	2	ego 坐标系， (x, y)
4	短期参考点 3	2	ego 坐标系， (x, y)
5	到参考中心线距离	1	标量距离
6	到左边界距离	1	标量距离
7	到右边界距离	1	标量距离

维度计算：

$$\begin{aligned} D_{\text{self}} &= 1 + 3 * 2 + 1 + 1 + 1 \\ &= 10 \end{aligned}$$

自车观测向量可以写成：

```
self_obs = [  
    ego_forward_velocity,  
    ref_1_x, ref_1_y,  
    ref_2_x, ref_2_y,  
    ref_3_x, ref_3_y,  
    distance_to_center_line,  
    distance_to_left_boundary,  
    distance_to_right_boundary,  
]
```

3.1 为什么没有自车位置和朝向

因为当前使用 ego-view。

在 ego 坐标系下：

自车位置 = (0, 0)
自车朝向 = 0

因此自车全局位置和自车朝向不需要作为输入。

如果切换为：

is_ego_view = false

自车位置和自车旋转才会进入观测。

4. 邻居车辆观测

邻居观测由 _observe_other_agents() 生成。

当前：

is_partial_observation = true
n_nearing_agents_observed = 3

因此每个 ego agent 只观测 3 个邻居。

当前邻居观测总维度：

D_neighbors = 33

每个邻居 11 维：

顺序	信息	维度	坐标系 / 说明
1	邻车顶点 1	2	ego 坐标系, (x, y)
2	邻车顶点 2	2	ego 坐标系, (x, y)
3	邻车顶点 3	2	ego 坐标系, (x, y)
4	邻车顶点 4	2	ego 坐标系, (x, y)
5	邻车速度	2	ego 坐标系下的速度表示
6	到邻车距离	1	标量距离

每个邻居维度：

```
D_one_neighbor = 4 * 2 + 2 + 1  
                = 11
```

3 个邻居：

```
D_neighbors = 3 * 11  
             = 33
```

邻居观测向量可以写成：

```
neighbor_obs_k = [  
    vertex_1_x, vertex_1_y,  
    vertex_2_x, vertex_2_y,  
    vertex_3_x, vertex_3_y,  
    vertex_4_x, vertex_4_y,  
    neighbor_vel_x, neighbor_vel_y,  
    distance_to_neighbor,  
]
```

完整邻居观测：

```
neighbors_obs = [  
    neighbor_1_obs, # 11 维  
    neighbor_2_obs, # 11 维  
    neighbor_3_obs, # 11 维  
]
```

5. 邻居顶点坐标系

当前邻居车辆 4 个顶点坐标是：

相对于 ego agent 的局部坐标

不是全局绝对坐标。

代码逻辑：

```

ver_i_others[:, a_i, a_j] =
    transform_from_global_to_local_coordinate(
        pos_i=pos_i,
        pos_j=self.vertices[:, a_j, 0:4, :],
        rot_i=rot_i,
    )

```

含义：

对 ego agent i 来说,
neighbor agent j 的四个角点会从全局坐标转换到 ego_i 的局部坐标系。

因此 actor 看到的邻居顶点是：

neighbor_j 的四个角点在 ego_i 坐标系下的位置

如果：

```
is_ego_view = false
```

邻居顶点才会使用全局坐标。

6. 对手建模动作占位

当前：

```
is_using_opponent_modeling = true
```

因此 actor 原始观测尾部会追加邻居动作占位：

```

D_opponent_tail = n_nearing_agents_observed * AGENTS["n_actions"]
                  = 3 * 2
                  = 6

```

每个邻居动作 2 维：

```
[v_command, steering_command]
```

尾部结构：

```
opponent_action_tail = [  
    neighbor_1_v_command,  
    neighbor_1_steering_command,  
    neighbor_2_v_command,  
    neighbor_2_steering_command,  
    neighbor_3_v_command,  
    neighbor_3_steering_command,  
]
```

重要区别：

- actor 使用时，这 6 维通常被置 0。
- critic 的 critic_observation 中，这 6 维会填入预测的邻居动作。

因此：

actor_observation 和 critic_observation 维度相同，都是 49；
但尾部 6 维的内容不同。

7. Actor 输入向量完整组成

当前每个 agent 的 actor 输入：

actor_obs: 49 维

完整展开：

```
actor_obs = [  
    # 自车观测, 10 维  
    ego_forward_velocity,  
    ref_1_x, ref_1_y,  
    ref_2_x, ref_2_y,  
    ref_3_x, ref_3_y,  
    distance_to_center_line,  
    distance_to_left_boundary,  
    distance_to_right_boundary,  
  
    # 邻居 1, 11 维  
    n1_v1_x, n1_v1_y,  
    n1_v2_x, n1_v2_y,  
    n1_v3_x, n1_v3_y,  
    n1_v4_x, n1_v4_y,  
    n1_vel_x, n1_vel_y,  
    distance_to_n1,  
  
    # 邻居 2, 11 维  
    n2_v1_x, n2_v1_y,  
    n2_v2_x, n2_v2_y,  
    n2_v3_x, n2_v3_y,  
    n2_v4_x, n2_v4_y,  
    n2_vel_x, n2_vel_y,  
    distance_to_n2,  
  
    # 邻居 3, 11 维  
    n3_v1_x, n3_v1_y,  
    n3_v2_x, n3_v2_y,  
    n3_v3_x, n3_v3_y,  
    n3_v4_x, n3_v4_y,  
    n3_vel_x, n3_vel_y,  
    distance_to_n3,  
  
    # 对手建模动作占位, 6 维  
    n1_v_command,  
    n1_steering_command,  
    n2_v_command,  
    n2_steering_command,  
    n3_v_command,  
    n3_steering_command,  
]
```

维度检查：

$$10 + 11 + 11 + 11 + 6 = 49$$

8. 归一化方式

观测进入网络前已经归一化。

当前关键归一化尺度：

信息	归一化尺度
ego-view 下的位置、 顶点、参考点	<code>[agent_length * 10, agent_length * 10] = [1.6, 1.6]</code>
速度	<code>max_speed = 1.0</code>
旋转角	<code>2π</code>
车道边界距离	<code>lane_width * 3</code>
参考线距离	<code>lane_width * 3</code>
agent 间距离	代码中当前写入 <code>past_distance_to_agents</code> 时使用 <code>distance_lanelet</code>
速度动作	<code>max_speed = 1.0</code>
转向动作	<code>max_steering_angle</code>

因此当前 actor 实际看到的是归一化后的数值。

例如邻居顶点：

$$\text{ego 局部坐标下的邻居顶点} / [1.6, 1.6]$$

边界距离：

$$\text{distance_to_boundary} / (\text{lane_width} * 3)$$